

Prompt Documentation

A structured record of the instructions given to the AI development assistant, the work each produced, and the rationale behind every design decision — from problem framing to a demo-ready prototype.

Prepared by	Harsh Goyal
Email	harshgoyal0807@gmail.com
Contact	+91 70935 23468
Project	Kohler Assist — unified enterprise AI agent for customers and the internal team (chat, phone and team console)
Deliverable	Prompt documentation (required submission artefact)
Scope	Complete development cycle: architecture, knowledge corpus, data model, build, evaluation, voice channel, operations layer
Method	Prompts are consolidated by phase and recorded in cleaned, professional wording that preserves the original intent; each is followed by the resulting work and the reasoning for the step
Data note	All customer, order and claim data in the prototype is fictional. Public documents are used for educational purposes only; internal policy documents were authored for the exercise and are marked fictional.

1. Purpose and method

This document is the prompt log for the Kohler Track 3 prototype. It serves two purposes: it is the transparency artefact the research program asks for, and it is the project's decision record — every architectural choice can be traced to the instruction that caused it and the reasoning that justified it.

HOW ENTRIES WERE RECORDED

- **Consolidated, not verbatim.** Instructions were given conversationally over several days, often as short follow-ups. Related instructions are merged into one professional prompt per milestone so that the log reads as a specification, not a transcript. Intent is preserved exactly; wording is cleaned.
- **Three parts per prompt.** *Prompt* (what was asked) → *Work produced* (what the assistant built or decided) → *Rationale* (why the step was taken and what it protects).
- **Grouped by phase.** Seven phases follow the natural order of the work. Within a phase, prompts are in the order they were given.
- **Excluded.** Housekeeping requests about this document itself, résumé help, presentation-deck drafting and routine "continue / status" nudges are not logged; where such nudges shaped the work they are folded into the relevant milestone.
- **Security.** No credential, key or token appears in this document; secrets live only in ignored configuration files.

2. Executive summary

The problem statement asked for an enterprise conversational agent over five knowledge bases (HR, finance, customer support, privacy, legal/compliance) with multi-turn reasoning and dynamic output formats (JSON schema, Excel, XML, ready-to-send email). The prompts in this log took that brief and shaped it into a system with four distinguishing properties:

- **Two audiences, two front doors, one brain.** External customers reach the agent on a real phone number (Sarvam AI voice agent, 11 Indian languages) or on the website; the internal team uses a knowledge chat and a ticket console. Both paths run the same Router → Retriever → Reasoner → Compiler pipeline against one knowledge base of 80 documents (57 public Kohler documents, 23 authored internal documents) and one customer record.
- **Data-sovereignty routing.** Every chunk carries a sensitivity label; access is enforced as a filter inside the vector database, and any answer that relies on restricted evidence is reasoned on a self-hosted model. An audit log makes this verifiable: zero restricted chunks reached a cloud model across 452 audited calls.
- **Evidence that shows its work.** Answers carry code-mapped citations; a version-conflict detector surfaces supersession — including two genuine revision chains found in public Kohler documents — with the vocabulary "superseded / review recommended / conflict", never "non-compliant".
- **Human-in-the-loop operations.** The agent prepares (claim intake, coverage check, photos, recommendation, drafts); a person decides in a shared ticket queue with the full case dossier and cited policy sources. Follow-ups by SMS and email and a shared memory let a customer move between phone and chat without repeating themselves.

The prototype was built on a CPU laptop in roughly two weeks of prompted development, evaluated with a 56-question internal harness, and verified with live inbound and outbound phone calls.

3. Prompt log by phase

Phase 1 — Problem framing and architecture	4 prompts
Phase 2 — Knowledge corpus: real documents, terms of use, OCR	5 prompts
Phase 3 — Data model: audiences, knowledge bases, regions, sensitivity	5 prompts
Phase 4 — Specification and build	4 prompts
Phase 5 — Evaluation and hardening	2 prompts
Phase 6 — Voice channel (Sarvam AI)	3 prompts
Phase 7 — Human-in-the-loop operations: claims, tickets, memory	5 prompts

Phase 1 — Problem framing and architecture

Establishing what to build before writing code: the architecture that would carry the 45 % innovation weight, the model strategy, and the routing rule that became the project's central idea.

PROMPT 1.1 — APPROACH TO THE PROBLEM STATEMENT

Given the Track 3 brief — an enterprise-grade conversational agent over HR, finance, customer-support, privacy and legal/compliance knowledge, with multi-turn reasoning and on-demand output in JSON schema, Excel, XML and draft-email form, judged 45 % on approach and innovation — propose how to approach it, then review a detailed plan (Router → Retriever → Reasoner → Response Compiler, LangGraph orchestration, FastAPI, hybrid BM25 + BGE-M3 retrieval in Qdrant with a reranker, metadata-based access control, cross-domain reasoning, contradiction detection, structured intermediate representation, ~50-question evaluation) and confirm full clarity before any coding starts.

Work produced. Proposed and then validated a Router → Retriever → Reasoner → Compiler architecture in place of a plain document chatbot: domain routing with persona-based access, hybrid retrieval with citations and contradiction detection, and a separate formatting stage driven by a structured intermediate representation (AnswerIR). Checked the development machine (CPU only, 16 GB RAM) and flagged the decisions that are expensive to reverse: fix the AnswerIR schema first, run embedding models via ONNX on CPU, engineer contradiction detection from version metadata rather than prompting, enforce access control as a vector-database payload filter, and split scope into must-ship versus stretch.

Rationale. With innovation weighted at 45 %, the architecture itself had to be the differentiator, and the choices that shape the data model had to be made before code existed.

PROMPT 1.2 — MODEL STRATEGY AND TIMELINE

Use the OpenAI API for general knowledge and a local open-source model for confidential material — a mixture of both. Target one to two weeks. Propose the departments to cover, with document counts and file formats, and use fastembed ONNX as the ML runtime.

Work produced. Formalised the hybrid model strategy as *data-sovereignty routing*: chunks carry a sensitivity tier and the reasoning model is chosen per answer, with an on-device badge in the interface when the self-hosted model is used. Produced a department-by-department corpus plan and a ten-day

build order. Re-read the problem statement and corrected four early deviations: sustainability is a criterion rather than a sixth knowledge base, internal-versus-external is an explicit access axis, XML is mandatory, and real public Kohler documents should replace synthetic ones wherever they exist.

Rationale. Turns the no-GPU, data-privacy constraint into the project's strongest feature and keeps the "unified platform over five knowledge bases" story exactly aligned with the brief.

PROMPT 1.3 — ROUTING RULE FOR TWO USER GROUPS

The application serves external customers and the internal team. Customer queries are processed on the OpenAI API; internal queries too, except where restricted information is involved, which must be handled by an open-source model. Restricted is the only tier that triggers the local model. Since documents are chunked and stored with metadata, route on the sensitivity of the retrieved chunks rather than on a query classifier.

Work produced. Settled the final rule after two refinements: a three-tier taxonomy (`public` / `internal` / `restricted`); retrieval runs fully on-device with the persona filter applied at the vector store; the language model is selected by inspecting the `sensitivity` of the retrieved chunks — any restricted chunk sends reasoning to the self-hosted model, otherwise to the cloud. A proposed query pre-classifier was dropped as redundant, making chunk metadata the single source of truth for both access control and model routing.

Rationale. The decision is driven by what was retrieved, not by who asked or how the question was phrased — a guarantee that can be audited rather than trusted.

PROMPT 1.4 — INTENT ROUTER AS THE FIRST STEP

Confirm that every query first passes through an intent router (an LLM call) before retrieval.

Work produced. Confirmed and fixed the five-node pipeline. The router is the first node because it decides whether to retrieve at all: *retrieval question* (with knowledge bases, region and cross-domain sub-queries), *format conversion* and *follow-up* (reuse the last `AnswerIR`, no retrieval), *action request*, or *out of scope*. The router sees only the query and chat history — never document text — so it may run on the cloud by default with a switch to the local model.

Rationale. The multi-turn formatting flow (answer → JSON → Excel → email) depends on distinguishing conversions from new questions; this fixed the pipeline that everything else was built on.

Phase 2 — Knowledge corpus: real documents, terms of use, OCR

Building a defensible corpus: genuine public Kohler documents wherever they exist, sourced within the publisher's terms, made fully text-readable, with the remaining gaps specified precisely.

PROMPT 2.1 — LOCATE REAL DOCUMENTS; INCLUDE SUSTAINABILITY AND INDIAN STANDARDS

Find the publicly available Kohler documents that can be downloaded. Include sustainability material. Add Indian standards (BIS certifications and ratings) alongside the product specification sheets, and cover both India-specific and global content, since the program builds global solutions for Kohler IT.

Work produced. Web-searched and HTTP-verified each source: supplier code of conduct, supplier terms, restricted-materials list, sustainability policy, Global Impact Report, WaterSense brochure, warranty

documents, eleven product specification sheets, live privacy notices, and Kohler India pages (warranty policies, privacy policy, terms, installation and refund guidance). Placed sustainability as a cross-cutting tag and later a customer-facing knowledge base, with a derived product catalogue so water-savings reports are computed from data. Mapped the Indian certification landscape (BIS/ISI mark, IS 17953:2023 water-efficiency star rating, IS 2556, IS 8931) and made `region` a first-class metadata dimension with a standards matrix and unit normalisation.

Rationale. Real documents give authentic page-level citations; a region- and standards-aware agent demonstrates exactly the global-solutions competence the program targets.

PROMPT 2.2 — MULTI-REVISION DOCUMENTS AND CAREFUL WORDING

The Global Supplier Quality Manual (quality-system requirements, certification preferences) is available — can it be used? And when the India privacy policy references the IT Act rather than the DPDP Act, does that imply non-compliance?

Work produced. Located four public revisions of the Supplier Quality Manual with verified differences — a genuine, unplanted test case for the contradiction detector. Clarified that no non-compliance is implied by the DPDP observation (rules phasing in; existing framework operative) and fixed the detector's vocabulary to `superseded` / `review_recommended` / `conflict`, never "non-compliant".

Rationale. Real revision chains are more convincing than planted conflicts, and the wording rule protects the project from making unsupported claims about the company.

PROMPT 2.3 — TERMS-OF-USE CHECK AND FINAL SOURCE LIST

Do a final sweep of what is publicly available and check Kohler's website terms and privacy policy to confirm whether the material may be downloaded and used. Then produce the final permitted list, fetch everything allowed, and label every synthetic document as fictional and for educational purposes only.

Work produced. Read the governing terms directly (download for personal, non-commercial use with notices retained; no redistribution; the India site additionally prohibits scraping and data mining) and set five sourcing rules: raw documents kept out of version control, one request per document with a hashed manifest, no crawlers, an attribution notice, and a private demo. Built `data/sources.yaml` (per-document domain, region, audience, sensitivity, version metadata and fetch method), a fetch script and a licensing notice; downloaded all scriptable documents and listed the few that required a manual browser save. Confirmed that no public finance documents exist because the company is privately held.

Rationale. Establishes a defensible data-sourcing policy before any document is collected and fixes the real-versus-fictional boundary for each knowledge base.

PROMPT 2.4 — CORPUS VALIDATION AND OCR

All remaining PDFs have been downloaded — scan the whole data folder and verify everything. Run OCR wherever needed so every document is usable as plain text; an LLM vision model may be used if local OCR proves insufficient.

Work produced. Scanned every file (opens, page count, text density, year, title). Discovered that the newly downloaded India warranty is a genuine newer revision (IN12-A → IN12-B; intelligent-toilet electronics

coverage 3 → 5 years) — the second real version-conflict test case. Dropped empty client-rendered pages, renamed manual downloads to canonical names, registered new documents with version and supersedes metadata. Only two scanned India CSR reports lacked a text layer; added a local OCR script (RapidOCR on CPU, reading-order reconstruction, per-page markers, word-segmentation repair) producing text sidecars with 0.80 mean confidence; LLM vision kept as an optional fallback.

Rationale. The ingestion pipeline should never meet a broken, empty or duplicate file, and a local, deterministic OCR fits the data-sovereignty design.

PROMPT 2.5 — INVENTORY AND SYNTHETIC-DOCUMENT PLAN

List the documents already held and specify exactly what still has to be generated.

Work produced. Inventoried the real documents per knowledge base and identified the gaps (no finance material; thin HR and internal procedures). Specified the synthetic set with region, format, audience, sensitivity and demo role: HR, finance, legal, privacy and support documents, a derived product catalogue, a small restricted subset for the on-device path, and planted version conflicts alongside the two real ones.

Rationale. Targets generation at demonstrable needs rather than volume, against a known real baseline.

Phase 3 — Data model: audiences, knowledge bases, regions, sensitivity

Making the access and region rules simple enough to explain in one sentence each, then generating the fictional documents that complete the corpus.

PROMPT 3.1 — GENERATE THE SYNTHETIC DOCUMENTS

Create the synthetic documents now. Put a fiction banner at the top of every document stating it is for educational purposes only and not real data; ground each one in the original Kohler documents; store them in the same folders as the originals rather than a separate synthetic folder; and maintain a reference spreadsheet listing every document as original or created by us.

Work produced. Built a generator with one block model rendered to PDF, DOCX, XLSX and Markdown, each carrying the red fiction banner on every page or sheet plus document code, version, effective date, classification and audience. Authored the internal documents grounded in the real corpus (gift rules follow the real Supplier Code; support service levels quote the real 90-day return rule and warranty IN12-B; the catalogue reuses real specification figures), including the planted version conflicts (India Leave Policy v1.0 → v2.0; India travel-and-expense addendum v1.4 → v2.1). Produced a manifest and `docs/Document_Inventory.xlsx` (Original versus Created-by-us), and validated that every file opens, carries the banner and has substantive text.

Rationale. Completes the corpus as one mixed knowledge base with a clear, auditable real/fictional boundary.

PROMPT 3.2 — FRONTEND CHOICE AND INDIA-FIRST REGION MODEL

Use React for the frontend. Clarify how domains and regions are handled: the product is effectively built for India, but many documents are global; all demo queries will be for India. Which important data was US-specific? Make the product specification sheets and the WaterSense brochure global. India is the child; the common parent data lives in Global.

Work produced. Switched to React + Vite. Audited the US-tagged material — only the specification sheets and the WaterSense brochure carried engineering data of general value — and re-tagged them global. Formalised a region inheritance model: `global` is the parent, `IN` the child that inherits and overrides; queries resolve child-first then parent; an Indian document overriding a global one is an override, not a contradiction, so the detector fires only on the same policy key with a different version. Rewrote the demo script so every step runs as an India persona.

Rationale. Aligns the product with its actual audience while keeping global-solutions capability, and prevents false conflict flags between parent and child documents.

PROMPT 3.3 — ONE SHARED INTERNAL KNOWLEDGE BASE

The internal knowledge base must not be personalised by role; every internal document is available to the whole internal team.

Work produced. Collapsed access control to two levels — Customer (public only) and Internal team (public + internal + restricted) — removing role gating. `audience` became `public | internal` across the manifest and regenerated documents; `restricted` remained purely a model-routing tier.

Rationale. Matches how a shared internal knowledge base actually works, simplifies the interface to two modes, and keeps exactly two enforced controls: the external/internal boundary and sovereignty routing.

PROMPT 3.4 — INTERNAL-ONLY VERSUS CUSTOMER-FACING KNOWLEDGE BASES; TWO REGIONS ONLY

HR, finance and legal are entirely internal. Privacy, support and reference are what customers can see; rename reference to sustainability. Remove the US and UK regions and keep only Global and India, folding genuinely important US or UK material into Global. Confirm the internal team can access all external documents.

Work produced. Reorganised the corpus into two sides: internal-only (`hr` , `finance` , `legal`) and customer-facing (`privacy` , `support` , `sustainability`), moving customer terms into support so legal is purely internal. Reduced the region model to `global` and `IN` ; parked US/UK documents outside the corpus, removed two US-only synthetic documents, and preserved the few important non-India facts inside global documents. Final corpus: 80 documents (57 real, 23 authored) in `<kb>/global` and `<kb>/in` folders; the internal team sees everything.

Rationale. Gives the knowledge base the same internal/external shape as the two user groups and removes an axis of complexity the India-first product did not need.

PROMPT 3.5 — RESTRICTED DOCUMENTS INSIDE CUSTOMER-FACING FOLDERS

Why do restricted documents sit in the external folders? Decision: keep them there.

Work produced. Explained that internal procedures about customer-facing topics (breach playbook, retention schedule, escalation matrix, recall procedure, tone guide, internal warranty copy) live beside the public policy so a single lookup returns both the policy and the handling rule, while `sensitivity` hides them from customers. Made the rule explicit: a folder is a topic, not a permission.

Rationale. Removes the ambiguity between folder and permission without changing retrieval behaviour.

Phase 4 — Specification and build

Freezing the specification, defining the value of the intermediate representation, adopting a proven chunking scheme, and building the complete prototype on CPU hardware.

PROMPT 4.1 — FREEZE THE SPECIFICATION

Before moving forward, finalise the technology stack, the approach to the problem statement, and the list of features and novelty. Document the agent's use cases and explain the AnswerIR.

Work produced. Wrote `docs/SPEC.md` as the single source of truth: requirement-to-solution mapping, the five-node pipeline with locked rules, the stack with rationale, chunk payload and persona filter tables, the `AnswerIR` schema, ranked novelty items, interface design for both modes, evaluation metrics with targets, the demo script and a build order. Explained `AnswerIR` — the validated, format-neutral structure of claims with claim-level sources, entities, conflicts with verdicts, access notes and summary — and catalogued eighteen use cases across customers and the internal team.

Rationale. Every decision frozen before implementation so code, interface and demo are built against one agreed specification.

PROMPT 4.2 — LIGHT AND DARK MODES; AN ACTIONS LAYER WITH A VOICE AGENT

Add light and dark modes. Add a simple "help with my product" flow: the customer enters order details and can either email the internal team or talk to an AI calling agent built on Sarvam AI that can act on the database and also handle orders and general queries. A Sarvam API key and a provisioned phone number are available; connect the Sarvam MCP servers so the agent can be built by instruction.

Work produced. Verified the platform capabilities (speech, dialogue model, voice-agent tooling, telephony, outbound calling, analytics) and configured the MCP servers with the key kept only in ignored configuration. Designed the actions layer: a "Help with my product" panel with email and voice channels; one shared brain in which the voice agent calls the same retrieval and the same guarded action tools (order lookup, warranty registration and check, claim, order, call-back) with an audit log; an inbox for the internal console; light/dark/system theming.

Rationale. Extends the agent from answering to acting, in the customer's language over a real phone line, while keeping one policy brain and the same evidence-and-audit discipline.

PROMPT 4.3 — CHUNKING STRATEGY

Adopt the chunking strategy used in the SecureGPT-v2 project.

Work produced. Adopted three chunk kinds — `prose` (section-based, small), `table_card` (one summary chunk per table) and `table_rows` (row groups with the header repeated and a "rows i-j of N" marker) — plus a context header on every chunk carrying knowledge base, region, document, version, effective date and section. Parsers emit structured tables from PDF, DOCX and XLSX; banner bands are stripped by position and tables are mapped to the heading above them. Result: about 1,500 well-formed chunks instead of roughly 1,950 fragments.

Rationale. Row-group chunks make spreadsheets and policy tables retrievable as units, and the header carries version context into the embedding for the detector and citations.

PROMPT 4.4 — BUILD THE COMPLETE PROTOTYPE ON CPU

Do not use rented GPUs; work on the CPU and develop the whole project through the night so that everything is finished by morning.

Work produced. Benchmarked and selected BGE-M3 via the official ONNX build with BM25 sparse vectors and a cross-encoder reranker, indexing demo-critical documents first; Ollama `qwen3:4b` for restricted reasoning with a capped evidence window. Built the full stack: ingestion, hybrid retrieval with persona and region filters and India precedence, the LangGraph pipeline (router → retrieve → gate → reasoner with version-conflict detection → compiler for text, schema-validated JSON, Excel, XML, table and email), streaming API, conversation memory for format conversions, the actions layer (SQLite CRM, guarded tools, audit log, voice endpoints), the React interface (customer and internal modes, pipeline strip, routing badge, conflict banner, citation panel, format switcher, help panel, inbox, themes), a 50-question evaluation harness with leak test, and run scripts. Verified end to end: paternity leave answered from v2.0 with the v1.0 supersession flagged; the salary-band question routed on-device; a claim opened from the help panel.

Rationale. Delivers the complete prototype on the hardware available while keeping every architectural commitment.

Phase 5 — Evaluation and hardening

Measuring the claims and making the scripted demo robust to model variance.

PROMPT 5.1 — RUN THE EVALUATION

Bring the backend and frontend up, run the full evaluation on the CPU, report progress, and keep a rented GPU as the fallback for the on-device model if latency becomes a problem.

Work produced. Ran the 56-question suite: 51 passed on the first full run — retrieval hit 49/49, customer-public-only 20/20, access 3/3, LLM-judged faithfulness 0.97, zero restricted chunks in any cloud call. Fixed the misses deterministically: a guard so "draft an email / reply" is always a rendering request, never a CRM action; version conflicts surfaced whenever either document is among the strongest evidence, not only when the small local model cites it; one expectation relaxed where restricted evidence legitimately pulls an answer on-device. A targeted re-run of the affected questions passed 8/8. No GPU was needed.

Rationale. The evaluation proves the sovereignty claim and exposed two demo-critical slips; fixing them deterministically makes the demo robust.

PROMPT 5.2 — FINAL CHECKS BEFORE RECORDING

Run final checks on every component and confirm the system is ready for the demo recording.

Work produced. Verified every service and walked the demo script through the API. Two content defects were found and fixed: the reasoner misread a flattened warranty table (quoting a generic three-year electronics row instead of the intelligent-toilet five-year row) — resolved by moving the reasoner to `gpt-5-mini` after a side-by-side comparison, with a table-reading rule; and the router fanned a warranty question out to a legal sub-query, diluting the evidence — resolved with a single-domain rule for support questions. All eight demo steps then passed.

Rationale. A scripted demo must not show a wrong number on stage; these were the last known sources of one.

Phase 6 — Voice channel (Sarvam AI)

Turning the phone into a full-capability front door: same knowledge, same rules, same record — in the caller's language.

PROMPT 6.1 — BUILD THE VOICE AGENT

The Sarvam voice-agents platform is connected — build the voice agent now.

Work produced. Created the agent (Hindi-first with language identification, product vocabulary, variables for call-back reason, phone and post-call summary), its HTTP tools into the backend (knowledge answer, order lookup, warranty check and registration, claim, order, call-back), a rule-based prompt (always use a tool; read back decisions; never discuss internal matters), and an inbound-plus-outbound deployment on the provisioned number with a call-end webhook into the inbox. Added a low-latency voice answer path (about eight seconds), model pre-warming, a webhook parser and the real outbound-call API. Documented two platform constraints discovered along the way: tool parameters created programmatically are static and must be switched to model-filled in the dashboard, and outbound calls need a platform API key.

Rationale. Completes the "one brain, two channels" promise: the phone reuses the same retrieval, rules and audit as the chat.

PROMPT 6.2 — LANGUAGES, LIVE TEST AND DIAL-IN

Commit the new version and place a test call. The agent must support every Indian language the platform offers, always answer in the language the caller is speaking, and translate English tool results naturally. Then verify that customers can dial in.

Work produced. Added greetings in all eleven supported languages, an explicit mirror-the-caller's-language rule and a parameter guide inside the prompt; moved the deployment to the new version and proved the chain over text, then placed a real outbound call. The 194-second call worked end to end — English answered in English, Hindi followed in Hinglish, order read back and confirmed, claim identifier spelled out, call-back placed, transcript and summary delivered by webhook. One defect (call-back sent to the number on the order rather than the caller's) was fixed. Inbound dialling was verified once the number was dialled in international format; a seven-minute inbound call loaded the caller's record from the caller ID, found the orders, completed a claim intake and delivered the recap.

Rationale. Turns the voice channel from "configured" into "demonstrated" on real telephony, in the caller's language.

PROMPT 6.3 — STABLE HOSTNAME AND TEAM REQUESTS FROM THE PHONE

The temporary tunnel has failed; re-point the tools without redoing manual dashboard work. Then deploy the version that lets callers send requests to the team.

Work produced. Replaced the random-hostname tunnel with a fixed-subdomain tunnel under a restart loop, updated the webhook and configuration, and had the tool hosts edited in place so the manual

parameter settings survived. Deployed the version with a team-request tool: the agent now confirms the caller's number before any order, call-back or team request, greets known callers by name in the first sentence, and a phone request was verified to create a product-query ticket for the right customer.

Rationale. A demo line must not depend on a hostname that changes on restart, and every request a caller makes must reach the same queue as a web request.

Phase 7 — Human-in-the-loop operations: claims, tickets, memory

Moving from automated decisions to automated preparation with human decisions, and making the two channels feel like one continuous conversation.

PROMPT 7.1 — NEVER AUTO-APPROVE; COMPLETE INTAKE; FOLLOW-UPS; MEMORY; LOGIN

The agent must never approve a claim, not even under the Tier-1 limit — every decision is made by a person; the bot's job is to ask all relevant questions, request photos and collect what the team needs. After a call or chat the customer receives an email or text with the actions taken and next steps. Both channels need real memory: a customer asked to upload photos and call back must be recognised and the conversation must resume on either channel. Add a customer login (mobile number plus a placeholder password) so questions work without signing in but every action requires it.

Work produced. Claims are filed as *pending* with a structured seven-field intake and a system recommendation for the reviewer; photos attach from the website or from a token-scoped upload page sent by SMS. A customer-memory record per mobile number is written after every action, upload, decision and call and read by both channels — the voice agent loads it at call start and resumes; the chat answers status questions from it without a language model. Follow-up SMS and email (simulated outbox, visible to the team and under the customer's "My messages") go out after calls, claims, uploads and decisions. Customer login by mobile number with a demo password; identity comes from the session, never the request.

Rationale. The compliance posture a real customer-care team needs — preparation automated, decisions human — while closing the follow-up loop across channels.

PROMPT 7.2 — DEMO CUSTOMERS, AN INTERNAL TEAM AND A SHARED TICKET QUEUE

Seed two named customers (one with older registered purchases, one with a recent unregistered purchase) and five internal employees. Every request to the internal team must be visible to the whole team; an employee picks a ticket and works it, with a dashboard over all tickets and full history. Any request to the team — including buying a product or asking for product information — must capture the customer's phone number.

Work produced. Seeded the customers and a five-person team with roles and tiers. Built a ticket engine: every claim, email, call-back, product query and order becomes a ticket in one shared queue; employees sign in, take tickets, add notes, reply to the customer (SMS + email, written into the customer memory), decide claims from the ticket and resolve; every step is an event and customer replies reopen the ticket. Added a tickets dashboard with status counters, breakdowns, filters, search, reassignment, priority and an SLA-age indicator; on the customer side an "Ask the team" request type and a "My requests" view. Requests, orders and team emails require the signed-in number.

Rationale. Turns the inbox into an operations workflow the team can actually run, with the customer's

identity attached to every request.

PROMPT 7.3 — CHECK THE POLICY FROM INSIDE A TICKET

Let a team member, from inside a ticket, ask the internal knowledge base about that order, read the policy sources, and only then act. Questions and answers should run as a conversation so follow-ups keep context, and the reply to the customer should be drafted from the whole thread.

Work produced. Added a threaded policy check inside each ticket: the specialist's question runs through the full internal pipeline with the case facts attached; the answer, version notes and expandable cited sources are shown and every turn is saved to the ticket history (and restored on reopening). A "draft reply from this thread" action writes the customer reply from all turns — on the self-hosted model whenever any turn touched restricted evidence. This rule exposed a labelling error (an everyday escalation matrix had been generated as restricted); it was reclassified, regenerated and re-indexed. Verified on a real claim: cited the India warranty policy and the escalation matrix and flagged the superseded revision.

Rationale. Evidence from the knowledge base is one click from the decision, and the evidence trail stays on the ticket.

PROMPT 7.4 — COMPLETE CASE CONTEXT, FOCUSED DRAFTS

When drafting, the assistant must have access to everything about the case — the request, details gathered on the call, the customer's history — but the message must be about the current issue on the ticket; past orders and history are reference only. A drafting request typed into the policy conversation should produce the draft there.

Work produced. Built a case dossier per ticket — request, claim with coverage window and intake, photos, recommendation, customer memory, recent call transcripts and ticket history — split into "current case" and "background (reference only)". Drafting receives the whole dossier, every research turn and the specialist's instruction and is told to write about the current issue only; policy questions receive a compact brief. Drafting requests typed into the thread are recognised and answered with the draft as a turn; a misrouting of such requests to the customer action path was fixed. The case view shows "what the assistant knows" in the two groups.

Rationale. Drafts are only trustworthy when they rest on the complete case yet stay on topic — and the team can see exactly what the assistant saw.

PROMPT 7.5 — CONSOLE STRUCTURE AND NAMING

The internal console should be a single "Tickets" area that keeps the existing inbox layout, with clickable summary counters; the queue must show its tickets after a restart; and the customer view must never expose the internal-team switch.

Work produced. Unified the console as "Tickets" (queue, claims, emails, call-backs, call transcripts, audit log) with clickable counters; opening a ticket shows a focused page with Case, Policy check and History tabs. The empty-queue defect was a login session lost on backend restart — sessions were moved to the database with an automatic re-prompt. The header shows only the current mode; changing mode goes through the landing page.

Rationale. Readability for the reviewer and a clean separation between what customers and employees can

see.

4. Decision register

The decisions that shaped the system, in the order they were locked. Each traces to a prompt in Section 3.

#	Decision	Prompt
D1	Architecture: Intent Router → local hybrid Retrieval (domain, persona, region filters) → sensitivity check on retrieved chunks → Reasoner with contradiction detector → Response Compiler; LangGraph orchestration on FastAPI.	1.1, 1.4
D2	<code>AnswerIR</code> is the contract between reasoning and rendering; output formats — text, schema-validated JSON, Excel, XML, Markdown table, email — are all must-ship and reuse the same IR across turns.	1.1, 4.1
D3	Data-sovereignty routing: three sensitivity tiers on every chunk; any <code>restricted</code> chunk in the evidence sends reasoning (and any draft derived from it) to the self-hosted model; otherwise the cloud model. No query pre-classifier; chunk metadata is the single source of truth.	1.2, 1.3
D4	Two audiences and two access levels enforced as a vector-database payload filter: Customer → public only; Internal team → all documents. No role-based gating inside the internal team.	1.3, 3.3
D5	Six knowledge bases on two sides: internal-only (HR, Finance, Legal) and customer-facing (Privacy, Support, Sustainability). A folder is a topic, not a permission; sensitivity decides per-document visibility.	3.4, 3.5
D6	Region model: <code>global</code> is the parent, <code>IN</code> the child that inherits and overrides; India documents take precedence; US/UK material excluded except where folded into global documents.	3.2, 3.4
D7	Corpus: real public Kohler documents (57) plus authored internal documents (23) marked fictional with a banner; sourced within the publisher's terms, one request per document, never redistributed, raw files kept out of version control.	2.1–2.5, 3.1
D8	Contradiction detection is engineered (same policy key, numeric version ordering) with the model only explaining differences; vocabulary <code>superseded</code> / <code>review_recommended</code> / <code>conflict</code> , never "non-compliant". Two genuine revision chains from public documents serve as test cases.	2.2, 2.4
D9	Retrieval stack: BGE-M3 (official ONNX) dense + BM25 sparse in Qdrant with RRF fusion and a cross-encoder reranker; SecureGPT-v2 chunking (prose, table card, table rows, context header); all on CPU.	1.2, 4.3, 4.4
D10	Models: <code>gpt-5-mini</code> for routing and reasoning (moved from <code>gpt-4.1</code> after it misread a flattened warranty table), <code>gpt-4.1-mini</code> for the voice fast path and the judge, Ollama <code>qwen3:4b</code> on-device.	4.4, 5.2
D11	Voice channel: Sarvam AI voice agent on a real number, 11 Indian languages with identification, tools calling the same backend, caller memory loaded at call start, call-end webhook into the team queue, outbound "call me now"; deployed and verified with live calls.	4.2, 6.1–6.3

D12	Human-in-the-loop: the system never approves or declines a claim; it runs a structured intake, requests photos, computes a recommendation, and a person decides in the shared ticket queue.	7.1, 7.2
D13	Identity and memory: customer sign-in by mobile number (placeholder password in the prototype), required for actions but not for questions; one customer-memory record per number shared by phone, chat, help panel and tickets; SMS and email follow-ups after every call, claim, upload, reply and decision.	7.1, 7.2
D14	Ticket operations: one shared queue for claims, emails, call-backs, product queries and orders; take, note, reply, decide, resolve with full event history; in-ticket policy check with a case dossier (current case versus background) and cited sources; drafts written from the whole research thread.	7.2–7.5

5. Outcome and measured results

WHAT WAS DELIVERED

- A working prototype on a CPU laptop: knowledge base of 80 documents (1,540 chunks) across six knowledge bases; Router → Retriever → Reasoner → Compiler pipeline with a sensitivity gate; customer web app (chat, help panel, sign-in), internal web console (knowledge chat, Tickets); SQLite actions layer; customer memory; simulated notifications.
- A Sarvam AI voice agent on a real Indian number with eight backend tools, 11 languages, memory at call start and a call-end webhook — verified with live inbound and outbound calls.
- Specification, demo checklist, voice-agent guide, document inventory, evaluation harness and this prompt log.

MEASURED ON THE PROTOTYPE (INTERNAL 56-QUESTION EVALUATION — 20 CUSTOMER, 36 INTERNAL — AND LIVE CALLS; NOT AN INDEPENDENT BENCHMARK)

- Leak test: **0 / 452** restricted chunks sent to any cloud model (all audited calls, voice and ticket checks included)
- Retrieval hit (expected document in the cited evidence): **49 / 49**
- Version-conflict pairs surfaced: **7 / 7** after fixes (first full run 6 / 7)
- Cloud answers: **13–25 s**; on-device answers: **≈ 2 min** (4B model on CPU; seconds on a GPU)
- Customer questions retrieving public chunks only: **20 / 20** (structural — filter enforced in the vector database)
- Faithfulness, LLM-judged over 43 answers: **0.97**
- Format conversion (JSON, Excel, XML, email) from an existing answer: **2–4 s**
- Voice: policy answer on a call **≈ 8 s**; a **7-minute** inbound call completed a full claim intake with memory loaded from the caller ID

Caveats stated deliberately: the question set and expectations were written by the project team against its own corpus; faithfulness is scored by a model, not by people; a single-question JSON-validity figure was withdrawn from headline use. The two guarantees the design stands on — no restricted content to the cloud, no internal content to customers — are enforced by architecture rather than measured by sampling.

Appendix — Glossary

Term	Meaning in this project
AnswerIR	Answer Intermediate Representation — the validated, format-neutral structure produced by the reasoner (claims with sources, entities, conflicts with verdicts, access notes, summary) that the compiler renders into any output format.
Persona	Who is asking: <i>customer</i> (external) or <i>internal</i> (employee / Customer Care). Travels with every request and is applied as a filter inside the vector database.
Sensitivity tier	Label on every chunk: <code>public</code> , <code>internal</code> or <code>restricted</code> . Public and internal govern access; restricted additionally forces on-device reasoning.
Sensitivity gate	The pipeline step that inspects the retrieved evidence and selects the cloud or the self-hosted model.
Leak test	Audit-log check counting restricted chunks that appeared in any cloud-model request; the design target is zero.
Version conflict	Two documents with the same policy key and different versions; the newer one is followed and the relationship is shown as superseded, review recommended or conflict.
Region model	<code>global</code> parent, <code>IN</code> child; the child inherits and overrides.
Case dossier	Everything known about a ticket — request, claim and coverage, photos, memory, call transcripts, history — split into current case and background.
Human-in-the-loop	The agent prepares (intake, checks, recommendation, drafts); a person makes every claim decision.

End of document.